DATA MINING · COURSE-BASED PROJECT · MASTER REFERENCE

Predicting a Movie’s Box-Office Fate Before Release

Hit/Flop classification from budgets, people, timing and expectations — and how much critic and viewer ratings add after release

STUDENT	Nadella Shyamanth
DEPARTMENT	Information Technology, VNR Vignana Jyothi Institute of Engineering & Technology, Hyderabad

COURSE	Data Mining — Course-Based Project (CBP), Academic Year 2026–27
SYSTEM	BoxOffice Oracle — React + TypeScript frontend, FastAPI backend, Python (pandas, scikit-learn, mlxtend) mining layer
PRIMARY DATASET	TMDB 5000 Movie Dataset (Kaggle) — movies + credits files
DOCUMENT	Version 2.0 (full-stack) · 22 October 2026 · the single source of reference for requirements, data, features, algorithms, execution, evaluation, report and demo

How to use this document: read Sections 1–7 for the design and the system; Sections 8–12 are the build guide (data, mining, models, API, UI); Sections 13–18 cover evaluation, report, demo and planning. Section 20 holds the tested data-preparation script.

Contents

1. Project at a Glance . . . 4
1.1 Problem statement . . . 4
1.2 Objectives . . . 4
1.3 Research questions . . . 4
1.4 Deliverables summary . . . 4
2. Background and Related Work . . . 5
2.1 Key papers . . . 5
2.2 Gap this project addresses . . . 6
3. Decisions Locked Before Any Work . . . 6
4. Data Sources and Requirements . . . 7
4.1 Primary dataset: TMDB 5000 Movie Dataset (Kaggle) . . . 7
4.2 Optional enrichment sources . . . 7
4.3 Required columns (data dictionary) . . . 7
5. Data Quality Rules . . . 8
6. Feature Engineering . . . 9
6.1 Feature catalogue . . . 9
6.2 Star power without leakage — the most important feature . . . 9
6.3 Leakage and the two-model design . . . 10
7. System Overview: BoxOffice Oracle . . . 10
7.1 What the user can do . . . 10
7.2 Architecture . . . 11
7.3 Technology stack . . . 11
7.4 Repository layout . . . 12
8. Execution Guide, Part 1: Data Pipeline . . . 12
8.1 Pipeline overview . . . 12
8.2 Environment setup (Windows) . . . 12
8.3 From script to package . . . 12
8.4 Single-movie input (what the app asks for) . . . 13
8.5 Validation checklist . . . 13
8.6 Exploration and preprocessing evidence (Python) . . . 13
9. Execution Guide, Part 2: Descriptive Mining . . . 14
9.1 OLAP engine . . . 14
9.2 Association rules: Apriori and FP-Growth (mlxtend) . . . 14
9.3 Clustering . . . 15
10. Execution Guide, Part 3: Prediction Models . . . 15
10.1 Training protocol . . . 15

10.2 Models . . . 15
10.3 Significance, importance and selection . . . 16
11. Backend API (FastAPI) . . . 16
11.1 Example request and response . . . 17
11.2 Backend requirements . . . 17
12. Frontend (React) . . . 17
12.1 Design and quality rules . . . 18
13. Evaluation Framework . . . 18
13.1 Metrics . . . 18
13.2 Comparison table (filled from artifacts/models/metrics.json) . . . 18
13.3 How to read the results . . . 19
14. Algorithm Reference Cards . . . 19
15. Build Phases and Syllabus Coverage . . . 20
16. Report Blueprint . . . 21
16.1 Limitations to state . . . 21
16.2 Future work . . . 22
17. Demo Plan . . . 22
17.1 Six-minute story . . . 22
17.2 Demo cases . . . 22
17.3 Prepared viva answers . . . 22
18. Plan, Team and Checklist . . . 23
18.1 Effort plan . . . 23
18.2 Roles for a group . . . 23
18.3 Master checklist . . . 23
18.4 Risk register . . . 24
19. References and Resources . . . 24
19.1 References (IEEE style) . . . 24
19.2 Dataset and tool links . . . 25
20. Appendix: Data-Preparation Script . . . 25
20.1 Glossary . . . 29

01 Project at a Glance

1.1 Problem statement

DO THIS

Predict, before a movie is released, whether it will be a Hit or a Flop at the box office — and measure how much critic and viewer reception adds to that prediction once the movie is out.

Studios commit most of a film's cost long before anyone sees it. If the information available at green-light and pre-release time — budget, genre, cast and director track record, franchise status, release window — carries a reliable signal about commercial success, a data-mining model can act as an early-warning system. Reception signals (ratings, vote counts) are known only after release; comparing a model with and without them quantifies how much of a movie's fate is already "decided" before opening day.

1.2 Objectives

1. Deliver a working web system that predicts Hit/Flop for any described movie, explains the prediction and supports what-if scenarios.
2. Build a clean, documented movie dataset (~3,000+ films after cleaning) with a defensible Hit/Flop label derived from return on investment.
3. Engineer pre-release features, especially cast and director track record computed *only from earlier films* (no leakage).
4. Apply every technique in the syllabus on the same data: preprocessing, OLAP, association rules (Apriori, FP-Growth), classification (ID3-style tree, C4.5/CART-style tree, Naive Bayes, KNN, Random Forest) and clustering (K-means, hierarchical).
5. Train two classifier families — Model A (pre-release) and Model B (pre-release + reception) — and compare them with 10-fold cross-validation against a majority-class baseline.
6. Explain which factors matter most and describe the natural segments of the movie market found by clustering.

1.3 Research questions

#	Question	Answered by
RQ1	Can pre-release information alone beat a naive baseline at predicting Hit vs Flop?	Model A vs baseline, Section 13
RQ2	How much do audience ratings and hype signals improve the prediction after release?	Model A vs Model B comparison table, Section 13
RQ3	Which attributes carry the most predictive weight (budget, star power, franchise, season…)?	Mutual-information ranking, decision tree, RF importance
RQ4	What combinations of attributes frequently co-occur with Hits or Flops?	Apriori / FP-Growth class-association rules
RQ5	What natural groups of movies exist, and how do their hit rates differ?	K-means + hierarchical clustering
RQ6	How do hit rates vary by year, genre, budget tier and season?	OLAP cube: roll-up, drill-down, slice, dice

1.4 Deliverables summary

Deliverable	What it contains	Where it comes from
Web application	React frontend (Predictor, What-if, Insights, Patterns, Models, Method) + FastAPI backend with tests	Sections 7, 11, 12

Deliverable	What it contains	Where it comes from
Dataset files	movies_clean.parquet/.csv, lookups, cleaning_log.txt	ml/ pipeline (Section 8, script in Section 20)
Data exploration	Attribute table, class counts, histograms, scatter plots (Insights page)	Section 8.6
Preprocessing evidence	Before/after counts for missing values, invalid money values, discretization, normalization	cleaning_log.txt, Section 8.6
OLAP	Cube definition; roll-up, drill-down, slice, dice and pivot outputs	Section 9.1
Association rules	Apriori and FP-Growth rules with stated support/confidence/lift settings	Section 9.2
Classifiers	ID3-style tree, C4.5/CART tree, Naive Bayes, KNN, Random Forest, baseline — for Model A and Model B	Section 10
Clustering	K-means (with elbow) and hierarchical results, each cluster interpreted	Section 9.3
Comparison	Accuracy, precision, recall, F1, AUC, kappa table: Model A vs Model B, every classifier	Section 13
Report + demo	Report following the college CBP template; 6-minute live demo of the app	Sections 16–17

02 Background and Related Work

Box-office forecasting is an established research problem. The papers below are the core references for the report's related-work chapter; each is summarised with what it means for this project.

2.1 Key papers

Study	Data and method	Finding	What we borrow
Sharda & Delen (2006), *Expert Systems with Applications* 30(2): 243–254	834 films released 1998–2002; neural network vs logistic regression, CART and others; 10-fold CV	Reframed forecasting as classification into nine revenue classes from "flop" to "blockbuster"; the neural net beat the alternatives. Reported ~37% exact-class accuracy and ~75% within one class.	Classification framing instead of exact revenue; 10-fold CV; pre-release variables (star value, MPAA rating, competition, genre, special effects, sequel, screens).
Delen & Sharda — comparative study of ML methods for Hollywood forecasting	Same nine-class setup; decision trees, ANN, logistic regression, plus sensitivity analysis	Variable-importance analysis on trained models adds interpretability.	Our feature-importance view (mutual information, decision tree, RF importance).
Lash & Zhao (2016), *Journal of Management Information Systems* 33(3): 874–903	2,506 films released 2000–2010; social-network and text features; "who, what, when" of a movie	Defined success as **profitability**, not raw revenue. Star power raises revenue but not necessarily profit, because stars are expensive; director track record mattered.	ROI-based target; cast/director history features; the "who/what/when" feature grouping.
Asur & Huberman (2010), arXiv:1003.5699	Tweet rate about 24 movies (~2.9M tweets) and sentiment; linear regression	Tweet-rate model outperformed the Hollywood Stock Exchange market forecast.	Motivates the "expectation/hype" signal — and its limitation: it is hard to measure before release.
Mestyán, Yasseri & Kertész (2013), *PLOS ONE* 8(8)	Wikipedia page views and edit activity for 312 US films (2010)	Predicted revenue up to a month before release with ~77% overall accuracy; better for big films.	Supports a pre-release "attention" feature as future work.

Study	Data and method	Finding	What we borrow
Zinoveva (2019), CEU MA thesis	Logistic regression on 4,727 films (1982–2016) using only public pre-release data	~62% correct classification; budget, franchise membership and US-company participation were important.	A realistic expectation: public-data pre-release models land in the 60–70% range — a credible benchmark.

NOTE
Set expectations in the report from this literature: with only public pre-release data, accuracies in the 60–70% range are normal. The project's contribution is not a record score — it is a clean, leakage-free comparison of pre-release vs post-release information, plus interpretable rules and segments.

2.2 Gap this project addresses

- Most studies predict revenue; fewer predict **profitability** with a transparent threshold.
- Few student-scale studies separate pre-release and post-release signals explicitly — our two-model design does.
- We combine predictive models with descriptive mining (OLAP, association rules, clustering) on one dataset, so every syllabus technique answers a business question.

03 Decisions Locked Before Any Work

These choices shape everything downstream. Write them into the report's introduction exactly as stated; changing one later means re-running the pipeline.

Decision	Locked choice	Justification to give in viva
Problem type	Binary classification: Hit vs Flop (3-class variant optional)	Matches the syllabus classifiers and Sharda & Delen's classification framing; easy to evaluate and explain.
Target source	ROI ratio = worldwide revenue ÷ production budget	Profit, not raw revenue, is what investors care about (Lash & Zhao). A ratio is roughly inflation-neutral because budget and revenue are from the same year.
Hit threshold	**Hit if revenue ≥ 2 × budget**, else Flop	Cinemas keep roughly half of ticket sales and marketing is not inside the production budget, so a film needs roughly 2× its budget to break even. 2× is the commonly used rule of thumb.
3-class variant	Flop < 1× · Average 1×–2.5× · Hit ≥ 2.5×	Optional extension; report it only if time allows.
Scope	Hollywood / mostly-US films in TMDB 5000 (spans 1916–2017)	Only source with reliable budget and revenue at this size. Indian-film budgets are rarely published consistently.
Year range	Use all valid rows; optional robustness run on 1980–2016 only	Keeps size up; the robustness run answers "aren't old films different?".
Model design	Model A = pre-release only; Model B = Model A + reception signals	Directly answers RQ1 and RQ2 and avoids leakage claims.
Evaluation	Stratified 10-fold cross-validation, random_state 1, majority-class baseline	Standard practice and used by Sharda & Delen.
Delivery	Full-stack web app: Python mining layer, FastAPI backend, React frontend	A usable system, not only offline experiments; every technique is visible in the UI.

WATCH OUT
If the course mandates an Indian-cinema scope, keep this exact pipeline but swap the data source; budgets must then be collected manually from trade sources for every row, which roughly doubles the data-collection effort. Decide this in week one.

04 Data Sources and Requirements

4.1 Primary dataset: TMDB 5000 Movie Dataset (Kaggle)

Collected from The Movie Database (TMDb) API and published on Kaggle as two files: **tmdb_5000_movies.csv** (title, budget, revenue, genres, release date, runtime, language, production companies, popularity, vote average, vote count, keywords...) and **tmdb_5000_credits.csv** (cast and crew lists). It holds about 4,800 films, mostly from the United States, released between 1916 and 2017.

```
Download:  https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata
Files   :  tmdb_5000_movies.csv   (4,803 rows, 20 columns)
           tmdb_5000_credits.csv  (4,803 rows: movie_id, title, cast, crew)
Join key:  movies.id  =  credits.movie_id
```

4.2 Optional enrichment sources

Source	Adds	Join	Use in
The Movies Dataset (Kaggle, Rounak Banik) — movies_metadata.csv, ~45,000 films released up to July 2017	**belongs_to_collection** → precise franchise flag; also IMDb IDs via links.csv	TMDB id	Franchise feature (the script uses it automatically if present)
Rotten Tomatoes Movies and Critic Reviews (Kaggle, scraped Oct 2020)	Tomatometer critic score, audience score	title + release year (fuzzy match; expect some losses)	Model B critic signal (optional)
IMDb non-commercial datasets (datasets.imdbws.com: title.basics, title.ratings)	IMDb average rating and number of votes	IMDb id (via links.csv)	Alternative audience-rating source for Model B
Box Office Mojo / The Numbers	Manual spot-checks of budget and revenue	manual	Validate 20 random rows; report the result

4.3 Required columns (data dictionary)

Column	Type	Source file	Role	Notes
id, title	id / text	movies	Identifier	Not used as model inputs
release_date	date	movies	Timing features + ordering for star power	Rows without a date are removed
budget	integer (USD)	movies	Target source + feature	0 or tiny values mean “unknown”
revenue	integer (USD)	movies	Target source only	Never used as an input (leakage)
genres	JSON list	movies	Descriptor	First genre = primary_genre; top 12 as 0/1 flags
runtime	minutes	movies	Descriptor	0 treated as missing → median
original_language	code	movies	Descriptor	is_english flag
production_companies	JSON list	movies	Descriptor	big_studio = any top-10 most frequent company
keywords	JSON list	movies	Franchise heuristic	“sequel”, “remake”, “based on comic”
cast	JSON list	credits	Star power	Top 3 billed actors (order field)
crew	JSON list	credits	Director track record	job == “Director”

Column	Type	Source file	Role	Notes
vote_average	0–10	movies	Audience rating — Model B only	Measured after release
vote_count, popularity	numeric	movies	Hype/engagement — Model B only	Snapshots taken in 2017, i.e. after release
status	text	movies	Filter	Keep “Released”

WATCH OUT

Critical honesty point: TMDb’s popularity and vote_count are snapshots taken when the dataset was collected (2017), long after most films were released. They reflect how much people engaged with a film *after* seeing it, so they are reception signals, not true pre-release hype. They therefore belong in Model B only. Say this explicitly in the report — examiners often ask it.

05 Data Quality Rules

Every rule is applied by the script and logged with before/after counts in **out/cleaning_log.txt**. Copy that log into the preprocessing chapter as evidence.

#	Issue	Rule	Why
R1	Duplicate movie ids	Keep first occurrence	Avoid double counting
R2	Unreleased titles (Rumored / Post Production)	Keep status == Released	No revenue exists yet
R3	Missing release date	Drop row	Needed for timing features and leakage-safe ordering
R4	Budget or revenue of 0 (unknown)	Drop row	Target cannot be computed; imputing a target is not acceptable
R5	Implausibly small values (e.g. budget = 1 or 7, meaning “millions”)	Drop rows with budget or revenue below US$10,000	These are data-entry shorthand; the threshold is stated and adjustable (MIN_MONEY)
R6	Runtime 0 or missing	Replace with the median runtime	Feature, not target — imputation is acceptable
R7	Extreme ROI outliers (micro-budget hits such as found-footage horror)	Keep, but report them; use log10(budget); check results with and without ROI > 50	They are real, not errors; a ratio-based label already limits their effect
R8	Stringified JSON fields	Parse with json / ast	Models need flat columns
R9	Newcomer actors/directors with no earlier film	Past revenue = 0, hit rate = overall prior, plus a count = 0 column	Keeps the row while letting the model know there is no history

NOTE

Expect roughly 3,200 rows to survive R1–R5 on the real data (many TMDB rows have a zero budget or revenue). Record the exact count from the log — do not quote this estimate in the report.

06 Feature Engineering

6.1 Feature catalogue

Feature	Type	How it is computed	Used in
log_budget	numeric	log10(budget) — tames the heavy right skew	A, B, Cluster
budget_tier	nominal	Tertiles of budget: Low / Medium / High	Rules, OLAP
runtime, runtime_bin	numeric / nominal	Minutes; bins Short (≤95), Standard (95–120), Long (>120)	A, B / Rules
primary_genre, n_genres	nominal / numeric	First listed genre; number of genres	A, B, Rules, OLAP
g_Action ... g_Thriller	0/1	12 genre flags (a film can have several)	CSV / optional extra runs
is_english	0/1	original_language == en	A, B
big_studio	0/1	Any production company among the 10 most frequent in the cleaned data	A, B, Rules
franchise	0/1	In a TMDb collection OR keyword sequel/remake/comic OR title contains 2, 3, II, Part…	A, B, Rules
release_month	numeric 1–12	Month of release date	A, B
season	nominal	Summer (May–Jul), Holiday (Nov–Dec), Dump (Jan–Feb), Regular (others)	A, B, Rules, OLAP
holiday_flag	0/1	Released in November or December	A, B
cast_avg_past_revenue	numeric	Mean revenue of all *earlier* films featuring any of the top-3 billed actors	A, B, Cluster
cast_past_hit_rate	0–1	Share of those earlier films that were Hits	A, B
cast_past_films	count	Number of earlier film appearances (experience)	A, B
dir_avg_past_revenue, dir_past_hit_rate, dir_past_films	numeric	Same three measures for the director	A, B, Cluster
star_tier, director_tier	nominal	Tertiles of the past-revenue measures	Rules
vote_average, rating_band	numeric / nominal	Audience rating; bands Poor (≤5.5), Mixed, Good, Great (>7.2)	B, Cluster / Rules
vote_count, popularity, hype_level	numeric / nominal	Engagement after release; hype_level = tertiles of vote_count	B, Cluster / Rules
roi_ratio	numeric	revenue ÷ budget	Target source; Cluster (descriptive only)
label (Hit/Flop), label3	nominal	roi_ratio ≥ 2 → Hit; 3-class variant	Class attribute

6.2 Star power without leakage — the most important feature

A naïve “average revenue of this actor’s films” uses the movie being predicted, and later films, to describe the actor. That leaks the answer into the input and inflates accuracy. The script prevents it:

1. Sort all cleaned films by release date.
2. Walk through the dates in order. For every film on a given date, compute the cast and director features using **only films already in the history** (strictly earlier dates).
3. Only after all films on that date have their features, add them to the history.

4. Actors or directors with no earlier film get past revenue 0, the overall hit rate as their prior, and a past-films count of 0.

```
history = {}                                  # person -> list of (revenue, is_hit)
for date in sorted(release_dates):
    for film in films_on(date):                # 1) features from strictly earlier films
        past = [h for actor in film.top3_cast for h in history[actor]]
        film.cast_avg_past_revenue = mean(revenue in past)   or 0
        film.cast_past_hit_rate    = mean(is_hit in past)    or prior_hit_rate
        film.cast_past_films       = len(past)
        (same for film.director)
    for film in films_on(date):                # 2) then add today's films to history
        for person in film.top3_cast + [film.director]:
            history[person].append((film.revenue, film.is_hit))
```

DO THIS

Viva line: “Every star-power value for a film is computed only from films released before it, so the model never sees its own outcome.” A small residual leak remains — the hit rate prior is computed on the whole dataset — mention it in limitations; its effect is negligible.

6.3 Leakage and the two-model design

Attribute group	Known when?	Model A	Model B	Rules	Cluster
Budget, genre, runtime, language, studio, franchise, month/season	Before release	✓	✓	✓	partly
Cast and director track record (earlier films only)	Before release	✓	✓	✓ (tiers)	✓
vote_average (audience rating)	After release	—	✓	✓ (band)	✓
vote_count, popularity (engagement)	After release (2017 snapshot)	—	✓	✓ (hype_level)	✓
revenue, roi_ratio	After release — it is the answer	—	—	— (label only)	descriptive only

07 System Overview: BoxOffice Oracle

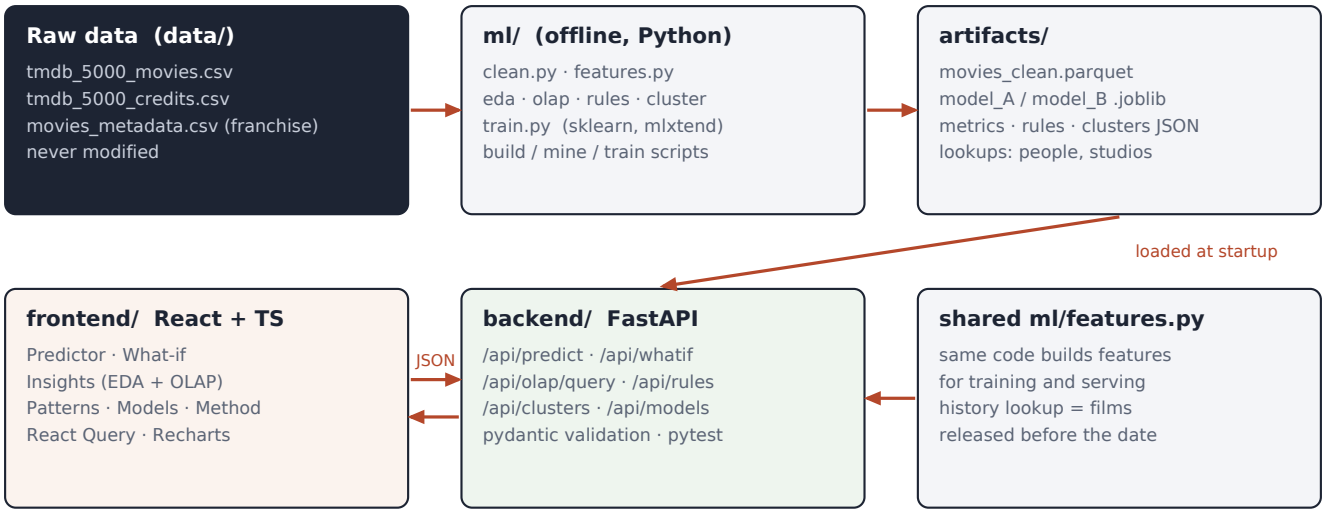
The project is delivered as a full-stack web system named **BoxOffice Oracle**. It predicts Hit/Flop for a movie described by its box-office-relevant facts, explains the prediction, runs what-if scenarios, and exposes every data-mining result (exploration, OLAP, association rules, clusters and model comparison) through an interactive interface. All mining runs in Python; the trained artefacts are served by a FastAPI backend to a React frontend.

7.1 What the user can do

Screen	What it does	Data-mining technique behind it
Predictor	Enter budget, genres, release date, runtime, language, studios, top-3 cast and director. Get Hit/Flop, a calibrated probability, the main factors behind it and any warnings. Optionally add rating, votes and popularity to switch to the post-release model.	Classification (Model A / Model B), feature engineering, star-power history
What-if simulator	Start from the form or a real movie and vary one factor (budget, release month, franchise, a cast member or the director). See the Hit probability for each value on a chart.	Repeated classification over a parameter sweep
Insights	Exploratory charts and an interactive OLAP explorer with roll-up, drill-down, slice, dice and pivot presets.	Data exploration, preprocessing evidence, OLAP

Screen	What it does	Data-mining technique behind it
Patterns	Filterable association-rule table with plain-English readings, the Apriori vs FP-Growth benchmark, cluster cards, elbow chart and dendrogram.	Apriori, FP-Growth, K-means, hierarchical clustering
Models	Model A vs Model B metrics, confusion matrices, feature importance, significance tests, the readable decision tree and the model card.	ID3-style tree, C4.5/CART-style tree, Naive Bayes, KNN, Random Forest, evaluation
Method	Data source and TMDb attribution, the Hit definition, leakage rules, two-model design and limitations.	—

7.2 Architecture



Two layers are deliberately separated. The **offline layer** (ml/) cleans data, mines patterns and trains models, writing everything to artifacts/. The **online layer** (backend/ + frontend/) only reads artefacts and builds features for new movies — it never retrains. The single shared module **ml/features.py** guarantees that a movie entered in the app is described exactly as the training rows were (training/serving parity), which a parity test proves.

7.3 Technology stack

Layer	Technology	Why
Data preparation	Python 3.11+, pandas, numpy	JSON parsing, leakage-safe star power, reproducible pipeline
Mining and models	scikit-learn, mlxtend (Apriori, FP-Growth), scipy (dendrograms, statistics)	Every syllabus algorithm is available and scriptable
Model storage	joblib + JSON artefacts	Fast load in the API; human-readable metrics
Backend	FastAPI, pydantic, uvicorn, rapidfuzz, pytest + httpx	Typed validation, auto docs at /docs, fuzzy name search, tests
Frontend	Vite, React, TypeScript, react-router, TanStack Query, Tailwind CSS, Recharts, react-hook-form + zod	Fast dev loop, typed forms, clean charts
Tooling	Antigravity agent with phase prompts; PowerShell scripts on Windows	Repeatable build from a fresh clone

7.4 Repository layout

```
boxoffice-oracle/
├── data/                  raw CSVs (read-only)
├── docs/                  this PDF, original prepare_movies.py
├── ml/
│   ├── clean.py           rules R1–R9 + cleaning log
│   ├── features.py        ONE feature builder: batch mode + single-movie mode
│   ├── build_dataset.py   writes artifacts/data/movies_clean.parquet
│   ├── eda.py · olap.py · rules.py · cluster.py
│   ├── train.py           models, CV, significance, importance, calibration
│   └── tests/             parity test, unit tests
├── artifacts/             data/ lookups/ eda/ olap/ rules/ clusters/ models/
├── backend/app/           main.py, routers/, schemas/, services/   + backend/tests/
├── frontend/src/          pages/, components/, api/, hooks/
├── scripts/               setup.ps1 build_data.ps1 mine.ps1 train.ps1 run_backend.ps1 run_frontend.ps1 start_all.ps1
├── report/                report draft, figures, slides outline, viva Q&A
├── PROJECT_CONTEXT.md     rules + phase log for the coding agent
└── README.md
```

08 Execution Guide, Part 1: Data Pipeline

8.1 Pipeline overview



MODEL A — pre-release (the real forecast)
budget · genre · runtime · language · studio · franchise
release month/season · cast & director track record

MODEL B — post-release (adds reception)
everything in Model A + vote_average (audience rating)
+ vote_count & popularity (hype) [+ critic score, optional]

8.2 Environment setup (Windows)

1. Python 3.11 or newer, a virtual environment in .venv, and `pip install -r requirements.txt` (pandas, numpy, scikit-learn, mlxtend, scipy, joblib, matplotlib, fastapi, uvicorn, pydantic, pytest, httpx, rapidfuzz, openpyxl).
2. Node.js LTS (18 or newer) and npm for the frontend.
3. Download TMDB 5000 (movies + credits) and movies_metadata.csv from Kaggle into data/.
4. Keep the original prepare_movies.py in docs/; the agent refactors it into ml/ without changing its logic.

8.3 From script to package

The tested script in Section 20 is the reference implementation. In the app it is split into modules so that the backend can reuse the feature code. The Weka-specific ARFF exports are dropped; the clean table is saved as Parquet and CSV.

Module	Responsibility	Output
ml/clean.py	Load, merge, apply rules R1–R9, write the cleaning log	Clean DataFrame + artifacts/data/cleaning_log.txt
ml/features.py	Batch mode: all features, star power from strictly earlier films. Single-movie mode: the same features for one new movie, looking up cast and director history in films released before its date.	Feature rows for training and for /api/predict

Module	Responsibility	Output
ml/build_dataset.py	Runs clean + features; builds lookups	movies_clean.parquet/.csv; lookups/people.json, studios.json, genres.json, languages.json
ml/tests/test_parity.py	Rebuilds 20 random films in single-movie mode and asserts equality with batch features	Proof of training/serving parity

8.4 Single-movie input (what the app asks for)

Field	Type / rule	Feeds
title	text, optional	Display only
budget	USD, > 0	log_budget, budget_tier
genres	1+ values from genres.json	primary_genre, n_genres, genre flags
release_date	date	release_month, season, holiday_flag, history cut-off
runtime	minutes, 40–300	runtime, runtime_bin
original_language	ISO code	is_english
production_companies	list	big_studio (top-10 list frozen at training time)
cast	up to 3 names	cast_avg_past_revenue, cast_past_hit_rate, cast_past_films, star_tier
director	name	dir_avg_past_revenue, dir_past_hit_rate, dir_past_films, director_tier
franchise	yes / no	franchise
vote_average, vote_count, popularity	optional	Present → Model B; absent → Model A

WATCH OUT
Unknown cast or director names are treated as newcomers (history count 0, prior hit rate) and the API returns a warning. History in the dataset ends in 2017, so a 2026 movie is scored against careers up to 2017 — the app says so.

8.5 Validation checklist

- ☐ Cleaning log saved; final row count and Hit/Flop split recorded
- ☐ Model A input list contains no vote, popularity, revenue or ROI attribute (printed as proof)
- ☐ Earliest films have cast_past_films = 0
- ☐ Avatar, The Dark Knight, Paranormal Activity, John Carter and The Lone Ranger checked by eye
- ☐ Parity test passes for 20 random films
- ☐ 20 random rows spot-checked against Box Office Mojo / The Numbers

8.6 Exploration and preprocessing evidence (Python)

Evidence	How	Shown in
Attribute table	type, missing, min, max, mean, std, top values per attribute	Insights page + report
Class distribution	Hit vs Flop counts and percentages	Insights page
Histograms	log budget, log revenue, ROI (clipped at 20, labelled), runtime, vote_average	Insights page
Hit rate by group	genre, season, budget tier, decade — with counts	Insights page

Evidence	How	Shown in
Missing values	runtime zeros imputed with median — before/after counts	Report (cleaning log)
Discretization	Tertile tiers and KBinsDiscretizer (quantile, 3 bins) — before/after distinct values	Report
Normalization	StandardScaler inside the KNN pipeline — before/after mean and std	Report

09 Execution Guide, Part 2: Descriptive Mining

9.1 OLAP engine

ml/olap.py exposes one pure function, `olap_query(measures, rows, cols, filters, level)`, over the clean fact table. The backend calls it live from `POST /api/olap/query`; the five classic operations are also precomputed as presets.

Element	Definition
Fact table	One row per movie (movies_clean)
Dimensions	Time: release_year → decade, month → season · Genre: primary_genre · Budget: budget_tier · Production: big_studio, franchise
Measures	count, hit_rate, avg_roi, total_revenue, median_budget
Guard rails	Count shown beside every rate; cells with fewer than 10 movies flagged low-confidence; unknown dimensions rejected with 422

Preset	Query
Roll-up	Hit rate by release_year, then by decade
Drill-down	2000s → years 2000–2009; season → month
Slice	primary_genre = Action → hit rate by budget_tier × decade
Dice	genre ∈ {Action, Horror, Comedy} ∧ season ∈ {Summer, Holiday} ∧ budget_tier ∈ {Low, High}
Pivot	genre × budget_tier and its transpose

```
def olap_query(df, measures, rows, cols=None, filters=None):
    d = df
    for dim, allowed in (filters or {}).items():          # slice / dice
        d = d[d[dim].isin(allowed)]
    agg = {"count": ("id", "count"), "hit_rate": ("is_hit", "mean"),
           "avg_roi": ("roi_ratio", "mean"), "total_revenue": ("revenue", "sum"),
           "median_budget": ("budget", "median")}
    keys = rows + (cols or [])                             # roll-up = fewer keys, drill-down = more
    out = d.groupby(keys).agg(**{m: agg[m] for m in set(measures) | {"count"}}).reset_index()
    out["low_confidence"] = out["count"] < 10
    return out                                             # pivot happens in the UI (rows <-> cols)
```

9.2 Association rules: Apriori and FP-Growth (mlxtend)

1. One-hot encode the nominal attributes: budget_tier, primary_genre, season, runtime_bin, star_tier, director_tier, franchise, big_studio, hype_level, rating_band, label.
2. Run `apriori` and `fpgrowth` with min_support = 0.03, then `association_rules` with metric = confidence, min_threshold = 0.65.
3. Keep rules whose consequent is exactly label=Hit or label=Flop (class association rules). Also build a **pre-release-only** set that excludes hype_level and rating_band.
4. Confirm both algorithms return identical rule sets; time each (mean of 3 runs) for the benchmark card.
5. Store antecedent, consequent, support, confidence, lift and a uses_post_release flag in artifacts/rules/rules.json.

> **NOTE**
> Interpret with lift: lift > 1 means the conditions make a Hit (or Flop) more likely than its base rate. Rules that use rating_band or hype_level are post-release and are labelled as such in the Patterns page.

9.3 Clustering

K-means

1. Features: log_budget, runtime, cast_avg_past_revenue, dir_avg_past_revenue, vote_average, vote_count, popularity — standardised. label and roi_ratio are excluded from fitting.
2. KMeans(init='k-means++', random_state=10, n_init=10) for k = 2…8. Save inertia (SSE) for the elbow chart and silhouette scores.
3. Choose k from the elbow and silhouette together (confirm it explicitly), then profile each cluster: size, centroid vs overall mean, hit share, median ROI.
4. Name every cluster from its numbers only.

Hierarchical

1. Take a reproducible sample of ~300 films (random_state = 1); standardise the same features.
2. scipy.cluster.hierarchy.linkage with Ward, average and complete; fcluster at the chosen k; save dendrogram data and images.
3. Compare with K-means on the same sample using the adjusted Rand index.

Cluster write-up template	
Name	Short label supported by the centroid
Size	Number and share of films
Centroid profile	Budget, star/director history, rating, engagement vs overall mean
Outcome	Hit share in the cluster vs overall hit share
Business reading	What a studio would do with this segment

10 Execution Guide, Part 3: Prediction Models

10.1 Training protocol

- Inputs come only from ml/features.py. Model A uses pre-release inputs; Model B adds vote_average, vote_count and popularity.
- Every model is an sklearn **Pipeline** (encoding, scaling, discretization, the estimator) so that preprocessing is fitted on training folds only.
- StratifiedKFold(n_splits=10, shuffle=True, random_state=1) for all comparisons.
- Class imbalance: if the minority class is under ~40%, compare with class_weight='balanced'; report per-class metrics either way.

10.2 Models

Model	scikit-learn implementation	Settings to report
Baseline	DummyClassifier(strategy='most_frequent')	—
ID3-style tree	KBinsDiscretizer(n_bins=3, strategy='quantile', encode='ordinal') → DecisionTreeClassifier(criterion='entropy')	sklearn has no exact ID3; this is the closest equivalent (entropy, all-categorical inputs). State this in the report.

Model	scikit-learn implementation	Settings to report
C4.5 / CART-style tree	DecisionTreeClassifier(criterion='entropy')	min_samples_leaf ∈ {2, 10, 25}
Naive Bayes	GaussianNB on numeric inputs; CategoricalNB on discretized inputs	Report both
KNN	StandardScaler → KNeighborsClassifier(weights='distance')	k ∈ {1, 3, 5, 7, 9, 11, 15}; pick by weighted F1
Random Forest	RandomForestClassifier(n_estimators=300, random_state=1)	max_depth ∈ {None, 8, 12} with inner CV

10.3 Significance, importance and selection

- **Significance:** 10 × 10-fold repeated stratified CV with the Nadeau–Bengio corrected resampled t-test (α = 0.05): each model vs baseline, and Model A vs Model B.
- **Importance:** mutual information ranking, Random Forest impurity importance and permutation importance, for both models.
- **Production choice:** best Model A and best Model B by weighted F1, then ROC AUC; refit on all data.
- **Calibration:** CalibratedClassifierCV (isotonic or sigmoid) so that “70% Hit” means roughly 70%; include a reliability plot.
- **Explanations:** an explain(movie) function returning the top factors for one prediction (per-prediction permutation deltas; SHAP only if approved).
- **Model card:** artifacts/models/model_card.json with inputs, rows, CV metrics, threshold, training date and limitations.

WATCH OUT
If Model A scores above ~90%, assume leakage until proven otherwise: re-check the Model A input list and that star power uses only earlier films.

11 Backend API (FastAPI)

Endpoint	Purpose	Notes
GET /api/health	Status, model versions, data row count	Used by the frontend banner
GET /api/meta	Genres, languages, studios, tier definitions, Hit threshold	Populates form options
GET /api/people/search?q=&role;=	Fuzzy autocomplete for actors and directors	rapidfuzz; returns film count and last film year
POST /api/predict	Predict Hit/Flop for a MovieInput	Model A, or Model B if reception fields are present; returns probability, factors, derived features, warnings
POST /api/whatif	Probability across values of one varying field	One result per requested value
GET /api/movies/search?q=	Load a dataset movie as a starting point	Returns its MovieInput and actual outcome
GET /api/insights/eda	EDA JSON	Precomputed
POST /api/olap/query · GET /api/olap/presets	Live OLAP and the five presets	Validated dimensions and measures
GET /api/rules?consequent=&prerelease;_only=&min;_lift=&sort;=	Filtered association rules	From rules.json

Endpoint	Purpose	Notes
GET /api/clusters	Profiles, elbow, silhouette, dendrogram data	From clusters JSON
GET /api/models	Metrics, significance, importance, model card, tree	From models JSON

11.1 Example request and response

```
POST /api/predict
{ "title": "Example Sequel", "budget": 150000000, "genres": ["Action","Adventure"],
  "release_date": "2016-06-17", "runtime": 128, "original_language": "en",
  "production_companies": ["Example Studio"], "cast": ["Actor A","Actor B","Actor C"],
  "director": "Director D", "franchise": true }

200 OK
{ "model": "A", "label": "Hit", "hit_probability": 0.71,
  "factors": [ {"feature": "franchise", "direction": "toward Hit", "weight": 0.09}, ... ],
  "derived": { "season": "Summer", "budget_tier": "High", "star_tier": "HighStar", ... },
  "warnings": ["Director D not found in history: treated as a newcomer",
               "Career history in the dataset ends in 2017"] }
```

(Values above are illustrative of the format only.)

11.2 Backend requirements

- Pydantic validation with clear 422 messages (budget > 0, sane date range, known language, at most 3 cast).
- Feature building calls ml/features.py — never reimplemented in the backend.
- Models and artefacts load once at startup; missing artefacts fail fast with a clear error.
- CORS limited to the frontend dev origin from .env; auto docs at /docs.
- pytest: schema validation; /predict equals a direct model call for a real dataset movie; /whatif returns one result per value; /olap/query rejects unknown dimensions; /people/search returns sensible matches.
- Target latency: /predict under 300 ms locally (report p50 and p95 over 50 calls).

12 Frontend (React)

Page	Main components	Key behaviour
Predictor	MovieForm (genres multi-select, cast/director autocomplete, franchise toggle, collapsible “After release” section), ResultCard (gauge, factors, derived features, warnings), LoadRealMovie	Filling the after-release section switches to Model B and the UI says so; real movies show the actual outcome beside the prediction
What-if	FactorPicker, ValueRange, ProbabilityChart	Baseline value highlighted; one factor at a time
Insights	EDA chart grid, OlapExplorer (rows, columns, measure, filters), PresetButtons	Each preset shows a one-line explanation of the OLAP operation
Patterns	RulesTable with filters, BenchmarkCard, ClusterCards, ElbowChart, Dendrogram	Plain-English reading per rule; post-release rules tagged
Models	ComparisonTable, ConfusionMatrix, ImportanceChart, SsignificanceSummary, TreeView, ModelCard	All numbers come from /api/models
Method	Static explainer sections	Includes the TMDb attribution line

12.1 Design and quality rules

- Dark cinematic theme with a light-theme toggle, one warm accent colour, generous spacing, readable type; works at laptop and phone width.
- Probabilities shown as percentages with a short plain-English sentence, never raw floats; budgets labelled USD.
- Every chart has a title, axis labels and units; loading, empty and error states on every page; backend error messages surfaced.
- No hard-coded numbers in the UI; the API base URL comes from an env file.
- Production build and TypeScript type check pass with zero errors.

13 Evaluation Framework

13.1 Metrics

Metric	Formula / meaning	Why report it
Accuracy	(TP + TN) ÷ all	Headline number; compare with the baseline
Precision (Hit)	TP ÷ (TP + FP) — of films predicted Hit, how many were Hits	Cost of backing a predicted hit that flops
Recall (Hit)	TP ÷ (TP + FN) — of actual Hits, how many were caught	Cost of passing on a real hit
F1	2 · P · R ÷ (P + R)	Single balanced score per class
Weighted-average F1	Per-class F1 weighted by class size	Main comparison metric across models
ROC AUC	Area under the ROC curve (roc_auc_score)	Threshold-free ranking quality
Kappa	Agreement beyond chance	Shows improvement over guessing
Confusion matrix	2×2 counts	Required in report for the best model

13.2 Comparison table (filled from artifacts/models/metrics.json)

Classifier	Model	Accuracy	Precision	Recall	F1	AUC	Kappa
Baseline	A						
Baseline	B						
ID3-style tree	A						
ID3-style tree	B						
C4.5/CART tree	A						
C4.5/CART tree	B						
Gaussian NB	A						
Gaussian NB	B						
Categorical NB	A						
Categorical NB	B						
KNN (k=…)	A						
KNN (k=…)	B						
Random Forest	A						
Random Forest	B						

13.3 How to read the results

- **RQ1:** Model A is useful if its best classifier clearly beats the baseline (aim for at least ~10 points of accuracy and a kappa well above 0) and the corrected t-test marks it significant.
- **RQ2:** The A → B difference is the "value of reception". A large jump means much of a film's fate is revealed only after release; a small one means pre-release information already captures most of it.
- **RQ3:** Agreement between mutual-information ranking, the decision tree's top splits and RF importance makes the importance claim robust.
- Report the winner by weighted F1 and AUC, not accuracy alone; explain the win with the algorithm's bias (Section 11).

14 Algorithm Reference Cards

ID3	Greedy top-down decision tree. At each node choose the attribute with the highest **information gain**.
KEY MATHS	Entropy $H(S) = -\Sigma\, p_i \log_2 p_i$; $Gain(S, A) = H(S) - \Sigma_v\, (\lvert S_v\rvert / \lvert S\rvert) \cdot H(S_v)$
CAVEATS	Nominal attributes only, no missing values, no pruning → overfits; biased toward many-valued attributes.
VIVA	Why discretize before an ID3-style tree? Why does ID3 overfit?
C4.5 / CART tree	Extension of ID3: uses **gain ratio**, handles numeric attributes with thresholds and missing values, and prunes with a confidence factor.
KEY MATHS	$GainRatio = Gain(S, A) \div SplitInfo(S, A)$; $SplitInfo = -\Sigma\, (\lvert S_v\rvert/\lvert S\rvert) \log_2(\lvert S_v\rvert/\lvert S\rvert)$
CAVEATS	In scikit-learn, min_samples_leaf and max_depth control tree size (C4.5 pruning is not reproduced exactly).
VIVA	Difference between ID3 and C4.5? What do depth/leaf limits do?
Naive Bayes	Probabilistic classifier using Bayes' theorem with the assumption that attributes are independent given the class.
KEY MATHS	$P(C \mid x) \propto P(C) \cdot \Pi_i\, P(x_i \mid C)$; numeric attributes modelled as Gaussian (or discretized / kernel)
CAVEATS	Fast, works well with little data; correlated features (budget and star revenue) violate the independence assumption.
VIVA	Why might NB still work despite correlated features?
KNN	Lazy learner: classify a film by the majority class of its k nearest training films.
KEY MATHS	Euclidean distance $d(x, y) = \sqrt{\Sigma\, (x_i - y_i)^2}$ on normalised attributes; optional 1/d weighting
CAVEATS	Needs normalisation; sensitive to irrelevant attributes and k; slow at prediction time.
VIVA	How did you choose k? Why normalise?
Random Forest	Ensemble of decision trees, each trained on a bootstrap sample with a random subset of attributes at each split; majority vote.
KEY MATHS	Bagging + random subspace; out-of-bag error; importance = mean impurity decrease
CAVEATS	Usually the most accurate here; less interpretable than one tree.
VIVA	Why does averaging many trees reduce variance?

Apriori	Finds frequent itemsets level by level, then derives rules above a confidence threshold.
KEY MATHS	support(X) = count(X)/N; confidence(X⇒Y) = support(X∪Y)/support(X); lift = confidence/support(Y). Downward closure: every subset of a frequent itemset is frequent.
CAVEATS	Multiple database scans and candidate generation; slow on many items.
VIVA	What is the Apriori property? What does lift > 1 mean?

FP-Growth	Compresses transactions into an **FP-tree** and mines frequent patterns recursively from conditional trees — no candidate generation.
KEY MATHS	Two scans: count items, then build the ordered tree; mine by suffix patterns
CAVEATS	Faster than Apriori on dense data; tree can be large in memory.
VIVA	Why is FP-Growth faster? Why one-hot encode before mining rules?

K-means	Partition n films into k clusters minimising within-cluster sum of squared errors (SSE).
KEY MATHS	Repeat: assign each point to nearest centroid; recompute centroids as means; stop when assignments stop changing. SSE = $\Sigma_k \Sigma_{x \in C_k} \|x - \mu_k\|^2$
CAVEATS	Needs k (elbow method); sensitive to initialisation (use k-means++) and to scale.
VIVA	How did you choose k? Why k-means++?

Hierarchical	Agglomerative: start with every film as a cluster and repeatedly merge the two closest clusters; the dendrogram shows the merge order.
KEY MATHS	Linkage: single (min distance), complete (max), average, Ward (minimum increase in variance)
CAVEATS	No k needed up front; $O(n^2)$ memory → sample the data.
VIVA	How do linkage choices change the clusters?

OLAP	Multidimensional analysis of a fact table over dimensions with hierarchies.
KEY MATHS	Roll-up (aggregate up a hierarchy), drill-down (go finer), slice (fix one dimension), dice (sub-cube on several), pivot (rotate)
CAVEATS	Descriptive, not predictive; depends on good dimension design.
VIVA	Difference between slice and dice?

15 Build Phases and Syllabus Coverage

The system is built with the Antigravity coding agent, one phase prompt at a time (the separate prompts file). Each phase ends with the agent reporting back before the next starts.

Phase	Builds	Syllabus techniques covered	Key outputs
1	Convert the project to full-stack; environment; dataset check	—	Layout, PROJECT_CONTEXT.md, requirements
2	Data pipeline and shared feature module	Data cleaning, missing values, discretization, feature engineering	movies_clean.parquet, lookups, parity test
3	Descriptive mining	Exploration, OLAP, Apriori, FP-Growth, K-means, hierarchical	EDA, OLAP, rules and cluster JSON
4	Prediction models	ID3-style tree, C4.5/CART tree, Naive Bayes, KNN, Random Forest, evaluation	model_A/B.joblib, metrics, significance, importance
5	FastAPI backend	—	API + tests

Phase	Builds	Syllabus techniques covered	Key outputs
6	React frontend	Visualisation of every technique	Six pages
7	Integration, demo cases, README	—	Setup scripts, latency numbers, demo cases
8	Report and slides	—	Report draft, figures index, PPT outline, viva Q&A

> **NOTE**
> If the lab record still needs weekly Weka screenshots, they can be produced separately from the same clean data; the app itself does not depend on Weka.

16 Report Blueprint

Follow the college CBP template for the front matter (certificate, declaration, acknowledgement, abstract, contents). Use the chapter plan below for the body.

Chapter	Contents	Source in this document
Abstract (150–200 words)	Problem, data, system, best Model A and Model B results, main finding	Fill last
1. Introduction	Motivation, problem statement, objectives, research questions, scope	Sections 1, 3
2. Literature survey	Papers table + gap	Section 2
3. Dataset	Sources, data dictionary, size, class distribution	Section 4
4. Preprocessing	Quality rules, cleaning log, discretization and normalization evidence	Sections 5, 8
5. Feature engineering	Catalogue, star-power algorithm, leakage table	Section 6
6. System design and implementation	Architecture, repository layout, training/serving parity, API, UI pages	Sections 7, 11, 12
7. Mining methodology	OLAP, association rules, clustering, classifiers, evaluation setup	Sections 9, 10, 13, 14
8. Results	Exploration, OLAP, rules, model comparison, clusters, importance, app screenshots	artifacts/ + app
9. Discussion	Answer each research question; compare with literature	Section 13.3
10. Limitations, conclusion, future work	Lists below	Section 16.1–16.2
References	IEEE style	Section 19
Appendix	Data-preparation script, API reference, extra screenshots	Section 20

16.1 Limitations to state

- **Leakage risk:** popularity and vote counts are post-release snapshots — kept out of Model A; residual leak from a global hit-rate prior.
- **Selection bias:** only films with published budget and revenue survive cleaning — typically larger, mainstream releases.
- **Hype is not fully measurable** with this dataset (no trailer views, searches or social-media activity before release).

- **Money measures:** revenue is worldwide gross, not studio profit; marketing spend is unknown; ticket prices change across decades.
- **Star power is dataset-internal** and stops in 2017 — new careers after that are invisible to the app.
- **Threshold sensitivity:** results depend on the 2× rule — show one robustness run at 1.5× or 2.5×.
- **ID3 approximation:** scikit-learn has no exact ID3; an entropy tree on discretized inputs is used.
- **Scope:** mostly Hollywood, ending 2017; streaming-era releases are not represented.

16.2 Future work

- Pre-release attention signals: Wikipedia page views, trailer views, search trends before release (Mestyán et al.).
- Refresh data from the TMDb API so that careers after 2017 are covered.
- Time-based validation (train up to 2012, test 2013 onward); revenue regression alongside classification; Indian-cinema extension with verified budgets.

17 Demo Plan

17.1 Six-minute story

Minute	Beat	Show in the app
0:00–0:45	Problem: studios bet big before release; can data warn them?	Method page header
0:45–1:45	Data: TMDB 5000 → cleaning → N films; ROI label; class split	Insights: class chart, attribute table
1:45–2:45	Patterns: OLAP preset and the best rules	Insights → OLAP dice preset; Patterns → top rules by lift
2:45–4:00	Live prediction on 3 real movies	Predictor → Load a real movie → compare with actual outcome
4:00–5:00	What-if: move the release month or swap the director	What-if chart
5:00–6:00	Model A vs B, segments, limitations	Models page + cluster cards

17.2 Demo cases

1. Choose 3 well-known dataset films: one big franchise release, one low-budget horror, one mid-budget drama.
2. Record each prediction, probability and actual outcome in report/demo_cases.md before the demo.
3. Rehearse with the backend and frontend started by scripts/start_all.ps1; record a screen video as a fallback.

17.3 Prepared viva answers

Likely question	Answer outline
Why ROI and not revenue?	Investors care about return; a big-budget film can gross a lot and still lose money. Lash & Zhao define success as profitability. A ratio is also roughly inflation-neutral.
Why 2× budget?	Theatres keep about half the gross and marketing isn't in the budget, so ~2× is break-even; a sensitivity run checks it.
Why can't Model A use ratings?	Ratings exist only after release — using them would be leakage. Model B shows how much they add.
How do training and serving stay consistent?	One shared ml/features.py builds features for both; a parity test proves identical output for 20 films.

Likely question	Answer outline
How did you avoid leakage in star power?	Features use only films released before the target film; history is updated after each release date.
Which model won and why?	Fill in from results; ensembles usually win because averaging trees reduces variance and captures interactions such as budget × franchise.
Why only ~60–70% accuracy?	Consistent with literature on public pre-release data (~62% in Zinoveva 2019); word of mouth and competition are not observable.
Is that really ID3?	scikit-learn has no ID3; we use an entropy-based tree on discretized inputs, the closest equivalent.
Apriori vs FP-Growth?	Same rules at the same thresholds; FP-Growth avoids candidate generation via the FP-tree, so it is faster — shown in the benchmark card.
What does the probability mean?	It is calibrated, so a 70% Hit prediction is right roughly 70% of the time on held-out data.
What if the actor isn't in the dataset?	Treated as a newcomer with a warning; history ends in 2017.

18 Plan, Team and Checklist

18.1 Effort plan

Phase	Share	Main outputs
Data pipeline and features	~25%	Clean data, lookups, parity test, validation
Descriptive mining	~15%	EDA, OLAP, rules, clusters
Models and evaluation	~20%	Model comparison, significance, calibration, explanations
Backend + frontend	~25%	API with tests; six pages
Report, slides and demo	~15%	Report, PPT, demo cases, rehearsal

18.2 Roles for a group

Role	Owns
Data + ML lead	ml/ package, parity test, models, model card
Mining lead	OLAP engine, rules, clustering, their report chapters
Backend lead	FastAPI endpoints, schemas, tests, latency
Frontend lead	Six pages, charts, design, build
Report + demo lead	Template, literature survey, PPT, demo script, viva sheet

18.3 Master checklist

- ☐ Problem statement, threshold and scope agreed
- ☐ Datasets in data/; sources recorded with dates; 20 rows validated
- ☐ Data pipeline runs; cleaning log saved; parity test passes
- ☐ EDA, OLAP presets, rules (Apriori + FP-Growth, identical) and clusters exported
- ☐ All models trained on Model A and Model B with 10-fold CV; comparison table filled
- ☐ Significance test, importance and calibration done; production models confirmed
- ☐ Backend endpoints pass tests; /predict under 300 ms
- ☐ Frontend pages build with zero type errors; no hard-coded numbers
- ☐ Robustness run (other threshold or 1980–2016 subset)

☐ README, report draft, slides and viva sheet complete

☐ Demo cases recorded; demo rehearsed and screen-recorded

18.4 Risk register

Risk	Likelihood	Mitigation
Kaggle download blocked on campus network	Medium	Download early; keep a copy in the shared drive
Training/serving mismatch (app predicts differently from CV)	Medium	Single features.py + parity test + /predict test
Name matching misses actors (spelling, accents)	Medium	rapidfuzz autocomplete; newcomer warning
Scores look “too good” (>90% on Model A)	Low	Leakage audit of inputs and star-power ordering
Frontend/back-end CORS or port issues on demo day	Medium	start_all.ps1; recorded video fallback
Examiner challenges the 2× threshold or the ID3 approximation	Medium	Sensitivity run; documented equivalence

19 References and Resources

19.1 References (IEEE style)

[1] R. Sharda and D. Delen, “Predicting box-office success of motion pictures with neural networks,” *Expert Systems with Applications*, vol. 30, no. 2, pp. 243–254, 2006.

[2] D. Delen and R. Sharda, “Forecasting financial success of Hollywood movies — a comparative analysis of machine learning methods,” SciTePress (conference paper).

[3] M. T. Lash and K. Zhao, “Early predictions of movie success: the who, what, and when of profitability,” *Journal of Management Information Systems*, vol. 33, no. 3, pp. 874–903, 2016, doi: 10.1080/07421222.2016.1243969. Preprint: arXiv:1506.05382.

[4] S. Asur and B. A. Huberman, “Predicting the future with social media,” arXiv:1003.5699, 2010.

[5] M. Mestyán, T. Yasseri and J. Kertész, “Early prediction of movie box office success based on Wikipedia activity big data,” *PLOS ONE*, vol. 8, no. 8, 2013.

[6] A. Zinoveva, “Can box office success be predicted before a movie’s release?” M.A. thesis, Central European University, 2019.

[7] J. Han, M. Kamber and J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed., Morgan Kaufmann, 2011.

[8] I. H. Witten, E. Frank, M. A. Hall and C. J. Pal, *Data Mining: Practical Machine Learning Tools and Techniques*, 4th ed., Morgan Kaufmann, 2016.

[9] J. R. Quinlan, *C4.5: Programs for Machine Learning*, Morgan Kaufmann, 1993.

[10] R. Agrawal and R. Srikant, “Fast algorithms for mining association rules,” Proc. VLDB, 1994.

[11] J. Han, J. Pei and Y. Yin, “Mining frequent patterns without candidate generation,” Proc. ACM SIGMOD, 2000.

[12] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, pp. 5–32, 2001.

[13] F. Pedregosa et al., “Scikit-learn: machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[14] S. Raschka, “MLxtend: providing machine learning and data science utilities and extensions to Python’s scientific computing stack,” *Journal of Open Source Software*, vol. 3, no. 24, 2018.

[15] M. Nadeau and Y. Bengio, “Inference for the generalization error,” *Machine Learning*, vol. 52, pp. 239–281, 2003.

19.2 Dataset and tool links

Resource	Link
TMDB 5000 Movie Dataset	kaggle.com/datasets/tmdb/tmdb-movie-metadata
The Movies Dataset (franchise flag)	kaggle.com/datasets/rounakbanik/the-movies-dataset
Rotten Tomatoes Movies and Critic Reviews	Search on kaggle.com (dataset scraped October 2020)
IMDb non-commercial datasets	developer.imdb.com/non-commercial-datasets
Box Office Mojo / The Numbers (validation)	boxofficemojo.com · the-numbers.com
scikit-learn	scikit-learn.org
mlxtend (Apriori, FP-Growth)	rasbt.github.io/mlxtend
FastAPI	fastapi.tiangolo.com
React + Vite	react.dev · vite.dev
Sharda & Delen 2006 abstract	scholars.okstate.edu (Oklahoma State University)
Lash & Zhao preprint	arxiv.org/abs/1506.05382
Asur & Huberman	arxiv.org/abs/1003.5699

Attribution line for the report (required by TMDb terms): “This product uses the TMDb API but is not endorsed or certified by TMDb.”

20 Appendix: Data-Preparation Script

This is the original reference script, tested end-to-end on a synthetic sample with the exact TMDB file structure. In the app it is refactored into ml/clean.py and ml/features.py without changing its logic; its ARFF exports (originally for Weka) are dropped. Place it at the project root and run `python prepare_movies.py`. Tunable constants are at the top.

```
"""
prepare_movies.py  -  Box-Office Hit/Flop project: data preparation pipeline
-----------------------------------------------------------------------------
Input  (put in ./data/):
    tmdb_5000_movies.csv      (Kaggle: TMDB 5000 Movie Dataset)
    tmdb_5000_credits.csv     (same dataset)
    movies_metadata.csv       (OPTIONAL, Kaggle: The Movies Dataset - for franchise flag)
Output (written to ./out/):
    movies_clean.csv          full cleaned table with every engineered feature
    modelA_prerelease.arff    Model A - only information known before release
    modelB_postrelease.arff   Model B - Model A + reception signals
    rules_nominal.arff        all-nominal table for Apriori / FP-Growth
    cluster_numeric.arff      numeric table for K-means / hierarchical clustering
    cleaning_log.txt          before/after counts for the report
Run:  python prepare_movies.py
"""
import ast, json, os, re
import numpy as np
import pandas as pd

DATA, OUT = "data", "out"
HIT_RATIO = 2.0          # Hit  <=> revenue >= 2 x budget
MIN_MONEY = 10_000       # budgets/revenues below this are treated as invalid (TMDB has 0, 1, 7 ...)
TOP_CAST = 3             # lead actors used for star-power features
os.makedirs(OUT, exist_ok=True)
log = []

def L(msg):
    print(msg); log.append(msg)

def parse_json_list(s, key="name"):
    """TMDB stores lists as stringified JSON, e.g. '[{"id": 28, "name": "Action"}]'."""
    if pd.isna(s) or s in ("", "[]"):
        return []
    try:
        items = json.loads(s)
    except (ValueError, TypeError):
        try:
            items = ast.literal_eval(s)          # movies_metadata.csv uses python-style quotes
        except (ValueError, SyntaxError):
            return []
    if isinstance(items, dict):
        items = [items]
    return [it.get(key) for it in items if isinstance(it, dict) and it.get(key) is not None]

# ---------------------------------------------------------------- 1. load + merge
movies = pd.read_csv(os.path.join(DATA, "tmdb_5000_movies.csv"))
credits = pd.read_csv(os.path.join(DATA, "tmdb_5000_credits.csv"))
credits = credits.rename(columns={"movie_id": "id"}).drop(columns=["title"], errors="ignore")
df = movies.merge(credits, on="id", how="left")
L(f"[load] movies={len(movies)} credits={len(credits)} merged={len(df)}")

# ---------------------------------------------------------------- 2. cleaning
n0 = len(df)
df = df.drop_duplicates(subset="id")
L(f"[clean] duplicates removed: {n0 - len(df)}")

if "status" in df.columns:
    n0 = len(df); df = df[df["status"] == "Released"]
    L(f"[clean] not-released removed: {n0 - len(df)}")

df["release_date"] = pd.to_datetime(df["release_date"], errors="coerce")
n0 = len(df); df = df[df["release_date"].notna()]
L(f"[clean] missing release_date removed: {n0 - len(df)}")

n_bad_budget = (df["budget"] < MIN_MONEY).sum()
n_bad_rev = (df["revenue"] < MIN_MONEY).sum()
n0 = len(df)
df = df[(df["budget"] >= MIN_MONEY) & (df["revenue"] >= MIN_MONEY)].copy()
L(f"[clean] invalid budget rows={n_bad_budget}, invalid revenue rows={n_bad_rev}, "
```

```
  f"removed together: {n0 - len(df)}  -> remaining {len(df)}")

df.loc[df["runtime"] <= 0, "runtime"] = np.nan
n_rt = df["runtime"].isna().sum()
df["runtime"] = df["runtime"].fillna(df["runtime"].median())
L(f"[clean] runtime missing/zero imputed with median: {n_rt}")

# ---------------------------------------------------------------- 3. target
df["roi_ratio"] = df["revenue"] / df["budget"]
df["label"] = np.where(df["roi_ratio"] >= HIT_RATIO, "Hit", "Flop")
df["label3"] = pd.cut(df["roi_ratio"], [-np.inf, 1.0, 2.5, np.inf],
                      labels=["Flop", "Average", "Hit"]).astype(str)
L(f"[target] Hit/Flop counts: {df['label'].value_counts().to_dict()}")
L(f"[target] 3-class counts: {df['label3'].value_counts().to_dict()}")

# ---------------------------------------------------------------- 4. descriptors
df["genres_list"] = df["genres"].apply(parse_json_list)
df["primary_genre"] = df["genres_list"].apply(lambda g: g[0] if g else "Unknown")
df["n_genres"] = df["genres_list"].apply(len)
TOP_GENRES = ["Action", "Adventure", "Animation", "Comedy", "Crime", "Drama", "Family",
              "Fantasy", "Horror", "Romance", "Science Fiction", "Thriller"]
for g in TOP_GENRES:
    df["g_" + g.replace(" ", "_")] = df["genres_list"].apply(lambda lst, g=g: int(g in lst))

df["companies"] = df["production_companies"].apply(parse_json_list)
top_companies = pd.Series([c for lst in df["companies"] for c in lst]).value_counts().head(10).index
df["big_studio"] = df["companies"].apply(lambda lst: int(any(c in top_companies for c in lst)))
df["is_english"] = (df["original_language"] == "en").astype(int)

df["release_year"] = df["release_date"].dt.year
df["release_month"] = df["release_date"].dt.month
def season(m):
    if m in (5, 6, 7): return "Summer"
    if m in (11, 12): return "Holiday"
    if m in (1, 2): return "Dump"
    return "Regular"
df["season"] = df["release_month"].apply(season)
df["holiday_flag"] = df["release_month"].isin([11, 12]).astype(int)

# ---------------------------------------------------------------- 5. franchise flag
df["keywords_list"] = df["keywords"].apply(parse_json_list) if "keywords" in df else [[]] * len(df)
meta_path = os.path.join(DATA, "movies_metadata.csv")
if os.path.exists(meta_path):
    meta = pd.read_csv(meta_path, low_memory=False, usecols=["id", "belongs_to_collection"])
    meta["id"] = pd.to_numeric(meta["id"], errors="coerce")
    meta["in_collection"] = meta["belongs_to_collection"].notna().astype(int)
    df = df.merge(meta[["id", "in_collection"]].drop_duplicates("id"), on="id", how="left")
    df["in_collection"] = df["in_collection"].fillna(0).astype(int)
    L("[franchise] collection flag taken from movies_metadata.csv")
else:
    df["in_collection"] = 0
    L("[franchise] movies_metadata.csv not found - using keyword/title heuristic only")
seq_kw = df["keywords_list"].apply(lambda k: int(any(w in ("sequel", "remake", "based on comic")
                                                     for w in k)))
seq_title = df["title"].str.contains(r"\b(?:2|3|4|5|II|III|IV|Part)\b", regex=True).astype(int)
df["franchise"] = ((df["in_collection"] + seq_kw + seq_title) > 0).astype(int)

# ---------------------------------------------------------------- 6. star power (NO leakage)
df["cast_list"] = df["cast"].apply(parse_json_list).apply(lambda c: c[:TOP_CAST])
def director_of(s):
    for p in (json.loads(s) if isinstance(s, str) else []):
        if p.get("job") == "Director":
            return p.get("name")
    return None
df["director"] = df["crew"].apply(director_of)

df = df.sort_values("release_date").reset_index(drop=True)
hist = {}                                   # person -> list of (revenue, is_hit)
cast_rev, cast_hit, cast_exp, dir_rev, dir_hit, dir_exp = ([] for _ in range(6))
for _, grp in df.groupby("release_date", sort=True):
```

```
    # compute features for every film on this date from STRICTLY earlier films ...
    for _, r in grp.iterrows():
        past = [h for a in r["cast_list"] for h in hist.get(("A", a), [])]
        cast_rev.append(np.mean([p[0] for p in past]) if past else np.nan)
        cast_hit.append(np.mean([p[1] for p in past]) if past else np.nan)
        cast_exp.append(len(past))
        dp = hist.get(("D", r["director"]), []) if r["director"] else []
        dir_rev.append(np.mean([p[0] for p in dp]) if dp else np.nan)
        dir_hit.append(np.mean([p[1] for p in dp]) if dp else np.nan)
        dir_exp.append(len(dp))
    # ... then add this date's films to history
    for _, r in grp.iterrows():
        rec = (r["revenue"], int(r["label"] == "Hit"))
        for a in r["cast_list"]:
            hist.setdefault(("A", a), []).append(rec)
        if r["director"]:
            hist.setdefault(("D", r["director"]), []).append(rec)
df["cast_avg_past_revenue"], df["cast_past_hit_rate"], df["cast_past_films"] = cast_rev, cast_hit, cast_exp
df["dir_avg_past_revenue"], df["dir_past_hit_rate"], df["dir_past_films"] = dir_rev, dir_hit, dir_exp
# newcomers: no history -> fill with 0 revenue / global prior hit rate, and keep the *_past_films=0 flag
prior = (df["label"] == "Hit").mean()
for c in ("cast_avg_past_revenue", "dir_avg_past_revenue"):
    df[c] = df[c].fillna(0)
for c in ("cast_past_hit_rate", "dir_past_hit_rate"):
    df[c] = df[c].fillna(prior)
L(f"[star] films with no cast history: {(df['cast_past_films'] == 0).sum()}, "
  f"no director history: {(df['dir_past_films'] == 0).sum()}")

# ---------------------------------------------------------------- 7. tiers / bins
def tertile(s, labels):
    return pd.qcut(s.rank(method="first"), 3, labels=labels).astype(str)
df["budget_tier"] = tertile(df["budget"], ["Low", "Medium", "High"])
df["runtime_bin"] = pd.cut(df["runtime"], [0, 95, 120, 1000], labels=["Short", "Standard", "Long"]).astype(str)
df["star_tier"] = tertile(df["cast_avg_past_revenue"], ["LowStar", "MidStar", "HighStar"])
df["director_tier"] = tertile(df["dir_avg_past_revenue"], ["LowDir", "MidDir", "HighDir"])
df["hype_level"] = tertile(df["vote_count"], ["LowHype", "MidHype", "HighHype"])        # post-release proxy
df["rating_band"] = pd.cut(df["vote_average"], [-1, 5.5, 6.5, 7.2, 11],
                           labels=["Poor", "Mixed", "Good", "Great"]).astype(str)
df["log_budget"] = np.log10(df["budget"])

# ---------------------------------------------------------------- 8. exports
keep = ["id", "title", "release_date", "release_year", "budget", "revenue", "roi_ratio", "label", "label3",
        "primary_genre", "n_genres", "runtime", "is_english", "big_studio", "franchise", "release_month",
        "season", "holiday_flag", "cast_avg_past_revenue", "cast_past_hit_rate", "cast_past_films",
        "dir_avg_past_revenue", "dir_past_hit_rate", "dir_past_films", "budget_tier", "runtime_bin",
        "star_tier", "director_tier", "hype_level", "rating_band", "vote_average", "vote_count",
        "popularity", "log_budget", "director"] + ["g_" + g.replace(" ", "_") for g in TOP_GENRES]
df[keep].to_csv(os.path.join(OUT, "movies_clean.csv"), index=False)

def q(v):
    """Make a nominal value ARFF-safe: 'Science Fiction' -> Science_Fiction (no quoting needed)."""
    return re.sub(r"[\s,{}'\"%]+", "_", str(v)).strip("_") or "Unknown"

def write_arff(frame, name, path, nominal):
    with open(path, "w", encoding="utf-8") as f:
        f.write(f"@RELATION {name}\n\n")
        for c in frame.columns:
            if c in nominal:
                vals = sorted(frame[c].astype(str).unique())
                f.write(f"@ATTRIBUTE {c} {{{','.join(q(v) for v in vals)}}}\n")
            else:
                f.write(f"@ATTRIBUTE {c} NUMERIC\n")
        f.write("\n@DATA\n")
        for row in frame.itertuples(index=False):
            f.write(",".join(q(v) if c in nominal else
                             ("?" if pd.isna(v) else f"{float(v):.4f}")
                             for c, v in zip(frame.columns, row)) + "\n")
    L(f"[arff] {path}: {len(frame)} rows x {len(frame.columns)} attributes")

A_cols = ["log_budget", "runtime", "n_genres", "is_english", "big_studio", "franchise",
```

```
          "release_month", "holiday_flag", "cast_avg_past_revenue", "cast_past_hit_rate", "cast_past_films",
          "dir_avg_past_revenue", "dir_past_hit_rate", "dir_past_films", "primary_genre", "season"]
A_nom = {"primary_genre", "season", "is_english", "big_studio", "franchise", "holiday_flag", "label"}
write_arff(df[A_cols + ["label"]], "movies_modelA", os.path.join(OUT, "modelA_prerelease.arff"), A_nom)

B_cols = A_cols + ["vote_average", "vote_count", "popularity"]
write_arff(df[B_cols + ["label"]], "movies_modelB", os.path.join(OUT, "modelB_postrelease.arff"), A_nom)

R_cols = ["budget_tier", "primary_genre", "season", "runtime_bin", "star_tier", "director_tier",
          "franchise", "big_studio", "hype_level", "rating_band", "label"]
rules = df[R_cols].copy()
rules["franchise"] = rules["franchise"].map({1: "Franchise", 0: "Original"})
rules["big_studio"] = rules["big_studio"].map({1: "BigStudio", 0: "Indie"})
write_arff(rules, "movies_rules", os.path.join(OUT, "rules_nominal.arff"), set(R_cols))

C_cols = ["log_budget", "runtime", "cast_avg_past_revenue", "dir_avg_past_revenue",
          "vote_average", "vote_count", "popularity", "roi_ratio"]
write_arff(df[C_cols + ["label"]], "movies_cluster", os.path.join(OUT, "cluster_numeric.arff"), {"label"})

with open(os.path.join(OUT, "cleaning_log.txt"), "w") as f:
    f.write("\n".join(log))
print("done ->", OUT)
```

20.1 Glossary

Term	Meaning in this project
ROI ratio	Revenue ÷ budget (2.0 = film earned twice its budget)
Leakage	Using information in the inputs that would not be available at prediction time
Model A / Model B	Pre-release-only classifier / classifier that also uses reception signals
Baseline	Majority-class classifier that always predicts the most common class
CAR	Class association rule — a rule whose right-hand side is the class label
Support / confidence / lift	How often a rule applies / how often it is right / how much better than chance
SSE	Within-cluster sum of squared errors (K-means objective)
Stratified 10-fold CV	Split data into 10 parts with the same class mix; train on 9, test on 1, rotate